ЛР1 Introductory Experiment

Учебник к защите

Вариант: **Write, No cache, 10M vs 1M, Rand**
Данилов Тимофей Николаевич, P3331

12.10.2026

КАК ЧИТАТЬ

Девять глав по плану подготовки, по порядку. В каждой: объяснение простыми словами, схема, пример с моими настоящими числами, блок «Строго» с точной формулировкой и вопросы с ответами в конце. Если на защите спросят «почему» — ответ почти всегда в блоке «Строго» соответствующей главы.

Все числа — из моей серии 12.10.2026 на машине `home` (i5-11400F, 31 ГиБ RAM, NVMe Kingston NV3, Ubuntu 24.04).

Содержание

1 Смысл лабы и вариант 4
1.1 Одной фразой 4
1.2 Модель данных: файл как лента вершин 4
1.3 Две программы 4
1.4 Ключи запуска 5
1.5 Мой вариант 5
1.6 Почему Write + NoCache + Rand — тяжёлое сочетание 5
1.7 Что такое эксперимент в этой лабе 6
1.8 Вопросы 6
2 Память: страницы и page fault 7
2.1 Виртуальная память 7
2.2 Страничный промах (page fault) 7
2.3 Порядки задержек 8
2.4 Вопросы 8
3 Ввод-вывод: системные вызовы, page cache, O_DIRECT 9
3.1 Системный вызов 9
3.2 Page cache: обычный путь 9
3.3 O_DIRECT: мимо page cache 9
3.4 Сброс кэша перед запуском 10
3.5 Вопросы 10
4 mmap 11
4.1 Что делает mmap 11
4.2 Запись через mmap 11
4.3 Что делает –no-cache у mmap 11
4.4 Почему mmap быстрее в тысячи раз 12
4.5 А если бы не было drop_caches или –no-cache 13
4.6 Вопросы 13
5 Программы и граф изнутри 14
5.1 Генерация графов 14
5.2 Цикл обхода read-версии 14
5.3 Одна вершина в read-версии 15
5.4 mmap-версия 15
5.5 Вопросы 15
6 Постановка эксперимента и инструменты 17
6.1 Паспорт системы 17
6.2 Окружение: что сделал и что было бы без этого 17
6.3 Инструменты 19
6.4 wall, user, sys 19
6.5 Вопросы 19
7 Статистика 21
7.1 Формулы 21
7.2 Разбор на моих числах: read-версия, 10M 21
7.3 s и ДИ — разные вещи 21
7.4 Почему t, а не 1,96 21
7.5 Сравнение конфигураций 22
7.6 Сколько нужно запусков 22

7.7 Прогрев и выбросы 22
7.8 Гистограмма и KDE 22
7.9 Вопросы 23
8 Мои результаты 24
8.1 Серия 24
8.2 Что видно 24
8.3 Откуда разброс: диск 24
8.4 Достаточно ли N = 10 25
8.5 Вопросы 25
9 Отчёт и защита 26
9.1 Что может попросить практик (README, раздел 8) 26
9.2 Команды для защиты 26
9.3 Самые вероятные вопросы 26

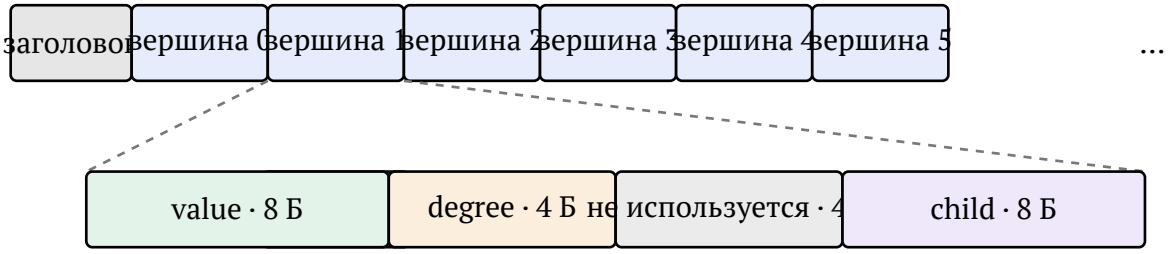
1 Смысл лабы и вариант

1.1 Одной фразой

Мы сравниваем, **как быстро программа проходит по записям в файле** при двух способах работы с файлом, и учимся **мерить это честно**: с контролем условий, серией запусков и статистикой.

1.2 Модель данных: файл как лента вершин

Файл графа — это заголовок и дальше подряд одинаковые записи по 24 байта. Каждая запись — вершина.



адрес вершины i = размер заголовка + 24 · i — находится за O(1), без поиска

Рис. 1. Формат файла графа. `value` — число, которое меняет режим `--write`; `degree` — есть ли следующая вершина; `child` — её номер.

Программа делает **обход цепочки**:

```
current = root
пока degree(current) > 0:
    прочитать вершину current            ← обращение к файлу
    если --write: value += 1, записать   ← ещё обращение к файлу
    current = child(current)             ← номер следующей берём из прочитанного
```

Это как связный список, только на диске. Главное свойство: **адрес следующей вершины становится известен только после чтения текущей**. Поэтому нельзя заранее «прочитать всё нужное одним куском» — программа не знает, что ей понадобится.

Зачем такая модель? Так ходят по данным базы данных и файловые системы: индексные деревья, связанные записи, журналы.

1.3 Две программы

	`graph_traverse` (read-версия)	**`graph_traverse_mmap`** (mmap-версия)
Открытие файла	`open()`; с `--no-cache` — с флагом `O_DIRECT`	`open()` + `mmap()` — файл становится массивом в памяти
Чтение вершины	`lseek()` встать на смещение, `read()` прочитать	просто чтение памяти `p[смещение]`
Запись вершины	`lseek()` + `write()`	`p[смещение].value++`
Когда работает ядро	на **каждый** вызов — сотни тысяч раз	на первом касании каждой страницы (page fault) и в конце (`msync`)

Логика обхода у них **одинаковая**. Различается только способ доступа к файлу. В этом суть лабы: «каждый раз просить ядро» против «один раз отобразить файл в память и работать как с массивом».

1.4 Ключи запуска

```
<программа> [--write] [--no-cache] <iterations> <файл>
```

- `--write` — увеличивать `value` каждой посещённой вершины и записывать обратно;
- `--no-cache` — не пользоваться системным кэшем файлов. У двух программ реализовано **по-разному** (главы 3–4);
- `iterations` — сколько раз пройти граф; у меня 1;
- `<файл>` — граф.

1.5 Мой вариант

Строка из журнала практика: `XS · Write,NoCache,10M vs 1M,Rand`.

Параметр	Значение	Что это значит
класс	XS	вводная обязательная лаба, самая маленькая по объёму
Read/Write	Write	обе программы запускаются с `--write`
Cache/No cache	No cache	обе программы запускаются с `--no-cache`
Size	10M vs 1M	два файла: 10 МиБ (436 905 вершин) и 1 МиБ (43 689 вершин)
Seq/Rand	Rand	соседние в цепочке вершины лежат на разных страницах файла

Изменяемый фактор один — размер файла. Всё остальное зафиксировано. Обе программы прогоняются на обоих размерах:

	1M (43 689 вершин)	**10M (436 905 вершин)**
read-версия	конфигурация A	конфигурация B
mmap-версия	конфигурация C	конфигурация D

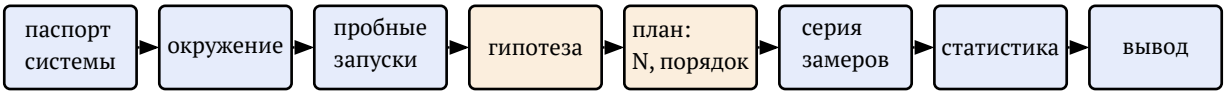
Что сравниваем:
1. **A ↔ B и C ↔ D** — как время зависит от размера файла. Это главный вопрос варианта.
2. **A ↔ C и B ↔ D** — read против mmap. Этого требует этап 3 README.

1.6 Почему Write + NoCache + Rand — тяжёлое сочетание

- **NoCache:** read-версия не может взять данные из памяти — каждое обращение идёт на диск.
- **Rand:** следующая вершина почти всегда на другой странице, соседние байты «за компанию» не помогают.
- **Write:** на каждой вершине ещё и запись — дополнительные обращения к диску.

Итог: у read-версии **каждая вершина — три операции с диском** (глава 5), а диск в тысячи раз медленнее памяти (глава 2). Поэтому обход 10M у неё длится ≈ 2,5 минуты, а у mmap-версии — ≈ 0,08 секунды.

1.7 Что такое эксперимент в этой лабе



гипотеза и план фиксируются ДО замеров

Рис. 2. Порядок работы по README.

СТРОГО — МОЖНО ГОВОРИТЬ НА ЗАЩИТЕ

Метрика запуска — время по часам (wall-clock) одного полного обхода графа. Результат конфигурации — оценка математического ожидания этого времени по серии из N запусков с 95-процентным доверительным интервалом. Гипотезы и правило отбрасывания прогревочных запусков фиксируются до получения результатов, чтобы исключить подгонку выводов под данные.

ПРИМЕР ИЗ МОИХ ЗАМЕРОВ

Одна и та же команда (read-версия, 1M) в моей серии дала `11,08 c`, `11,11 c` и `25,34 c`. По одному запуску можно было бы получить любое из этих чисел и сделать противоположные выводы.

1.8 Вопросы

В: Чем отличаются две программы?
О: Логика обхода одна. read-версия на каждую вершину делает системные вызовы `lseek`/`read`/`write`, mmap-версия отображает файл в память и обращается к вершинам как к массиву; ядро участвует только при первом касании страницы.

В: Что изменяется в эксперименте?
О: Только размер файла: 1 МиБ и 10 МиБ. Режимы `--write` `--no-cache` и топология Rand фиксированы.

В: Зачем гипотеза до замеров?
О: Чтобы не подогнать «ожидание» под полученные числа. Эксперимент гипотезу подтверждает или опровергает; опровержение — тоже результат.

В: Почему нельзя прочитать файл одним куском?
О: Адрес следующей вершины известен только после чтения текущей. Сам файл в RAM влезает (10 МиБ ≪ 31 ГиБ), и без `--no-cache` page cache так бы и сделал — но вариант запрещает кэш.

2 Память: страницы и page fault

2.1 Виртуальная память

Каждый процесс живёт в своём **виртуальном адресном пространстве**: ему кажется, что у него вся память от 0 до огромного адреса. На деле физическая RAM раздаётся кусками по **4 КиБ — страницами**. Ядро ведёт для процесса **таблицу страниц**: «виртуальная страница № X лежит в физической рамке № Y» или «пока нигде».

Перевод адреса делает сам процессор — блок **MMU**. Чтобы не читать таблицу при каждом обращении, последние переводы он держит в маленьком кэше **TLB**.

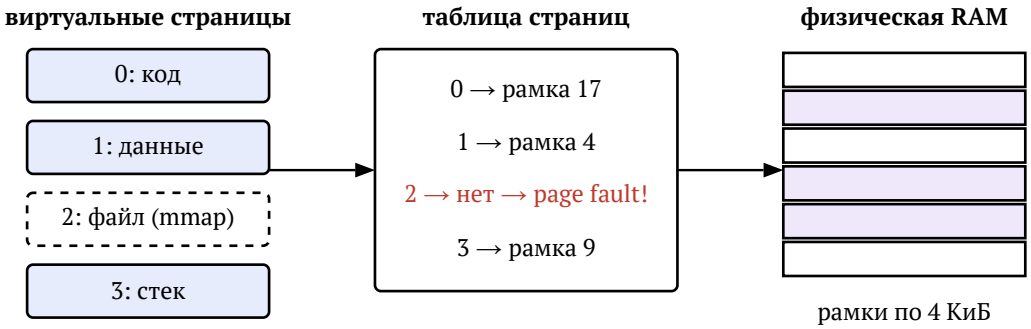


Рис. 3. Перевод адреса. Для страницы 2 отображения ещё нет — обращение к ней вызовет страничный промах.

Адрес делится на две части: номер страницы и смещение внутри неё. Для 4 КиБ смещение — младшие 12 бит:

```
виртуальный адрес 0x7f3a_1234_5678
  номер страницы = 0x7f3a_1234_5   (переводится через таблицу)
  смещение       = 0x678           (остаётся как есть, 0..4095)
```

2.2 Страничный промах (page fault)

Если для страницы в таблице нет записи (или права не позволяют, например запись в страницу «только для чтения»), процессор останавливает инструкцию и передаёт управление ядру. Это **страничный промах**. Ядро разбирается, что случилось:

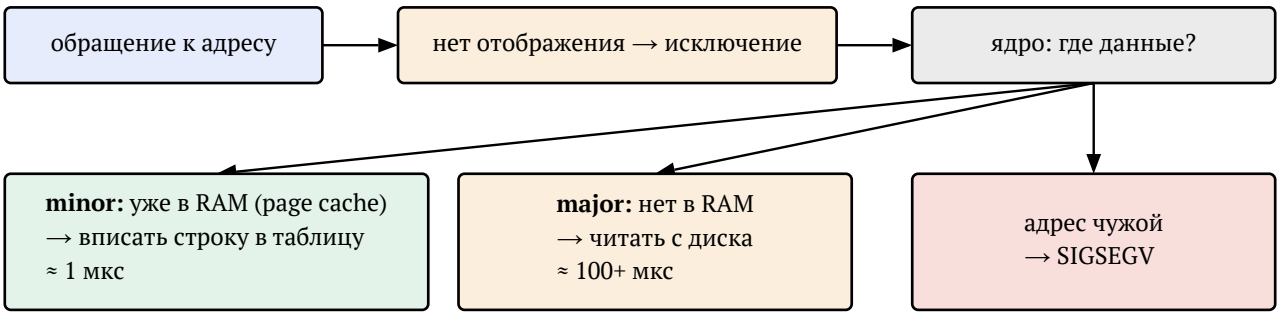


Рис. 4. Три исхода страничного промаха.

После minor или major fault ядро возвращает управление, процессор **повторяет ту же инструкцию**, и теперь она проходит. Программа ничего не замечает, кроме потерянного времени.

СТРОГО — МОЖНО ГОВОРИТЬ НА ЗАЩИТЕ

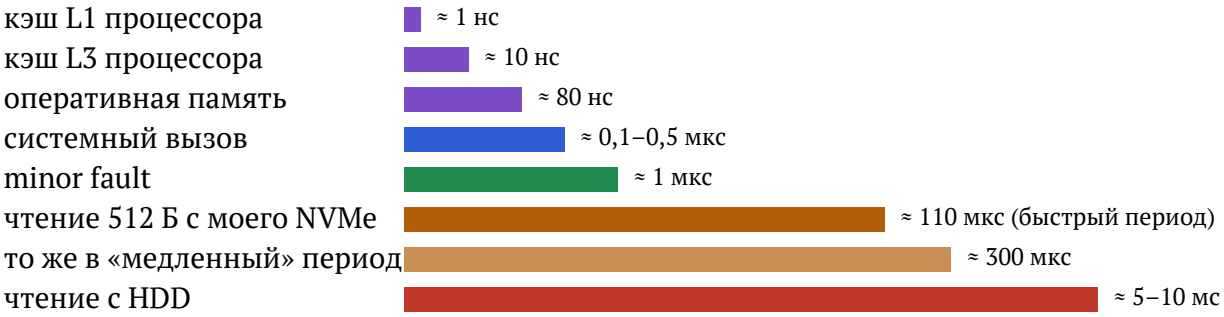
Страничный промах — синхронное исключение процессора при обращении к странице, для которой нет действительного отображения в таблице страниц или права доступа не допускают операцию. Обработчик ядра устанавливает отображение на уже резидентную рамку (minor fault), инициирует ввод-вывод для загрузки страницы с носителя (major fault) либо посылает процессу SIGSEGV. После обработки инструкция выполняется повторно.

ПРИМЕР ИЗ МОИХ ЗАМЕРОВ

`/usr/bin/time -v` у mmap-версии на 10M: **major faults ≈ 208, minor faults ≈ 2913**. У read-версии на тех же 10M: major ≈ 2, minor ≈ 221 — это промахи при загрузке самой программы и библиотек, к файлу графа они отношения не имеют: read-версия читает файл не через память, а системными вызовами.

2.3 Порядки задержек

Числа, которые надо держать в голове:



Шкала логарифмическая: каждый шаг вправо — примерно в 10 раз дольше.

Вывод: **поход к диску в тысячи раз дороже всего, что делает процессор**. Поэтому время read-версии в моём варианте — почти целиком ожидание диска, а не вычисления.

2.4 Вопросы

В: Что такое страница?
О: Единица, которой ядро управляет памятью; на x86-64 — 4 КиБ.

В: Чем minor fault отличается от major?
О: Minor — страница уже в RAM, нужна только запись в таблице страниц. Major — страницу надо прочитать с носителя.

В: Что такое TLB?
О: Кэш процессора для переводов виртуальных адресов в физические.

В: Почему у read-версии почти нет page faults?
О: Она не отображает файл в память: данные копируются ядром в буфер программы, выделенный один раз.

3 Ввод-вывод: системные вызовы, page cache, O_DIRECT

3.1 Системный вызов

Программа не может сама обратиться к диску — это делает только ядро. Чтобы попросить ядро, программа выполняет **системный вызов**: специальную инструкцию (`syscall` на x86-64), которая переключает процессор в режим ядра. Ядро выполняет просьбу и возвращает управление.

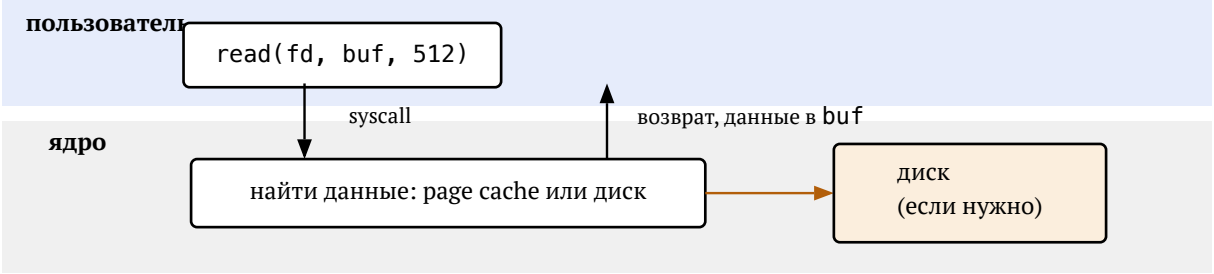


Рис. 5. Системный вызов: переход в ядро и обратно. Сам переход стоит ≈ 0,1–0,5 мкс; если нужен диск — плюс задержка диска.

3.2 Page cache: обычный путь

Ядро держит прочитанные куски файлов в свободной RAM. Это **page cache**. Его видно в `free -h`, колонка `buff/cache`.

- **Чтение.** Первый `read()` места в файле идёт на диск, данные кладутся в page cache и копируются в буфер программы. Повторный `read()` того же места диск не трогает — копия из памяти.
- **Запись.** `write()` по умолчанию только меняет страницу в page cache и помечает её **грязной (dirty)**. На диск её позже, в фоне, запишет механизм **writeback** (обычно через десятки секунд). Программа к этому моменту может уже завершиться.
- **Readahead.** Если ядро видит последовательное чтение, оно заранее читает следующие страницы.

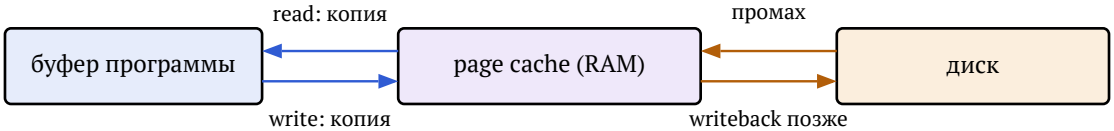


Рис. 6. Буферизованный ввод-вывод: программа общается только с page cache, диск — с page cache.

3.3 O_DIRECT: мимо page cache

`O_DIRECT` — флаг при `open()`: «передавай данные прямо между моим буфером и диском, page cache не используй». Каждое `read()` и `write()` превращается в реальную операцию с устройством.

Цена: диск работает **блоками**. Поэтому при `O_DIRECT` адрес буфера, смещение в файле и длина должны быть **кратны размеру логического блока**. У моего NVMe это 512 байт (`/sys/block/nvme0n1/queue/logical_block_size`). Если не выровнять — вызов вернёт ошибку `EINVAL`.

Вершина занимает 24 байта и лежит где-то внутри блока. Поэтому программа (файл `src/graph_io.h`, функция `graph_direct_transfer`):

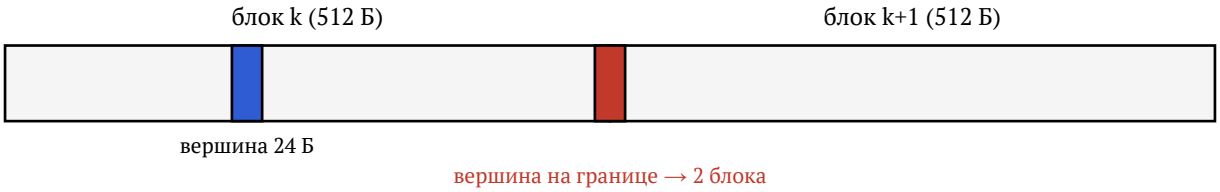


Рис. 7. Выравнивание. Обычная вершина целиком в одном блоке; примерно 4–5% вершин пересекают границу и требуют двух блоков.

- **чтение вершины:** прочитать целый блок 512 Б, вырезать из него 24 байта;
- **запись `value`: read-modify-write** — снова прочитать блок, заменить в нём 8 байт, записать блок целиком. Перечитывание нужно, чтобы не затереть соседние вершины в том же блоке.

> **СТРОГО — МОЖНО ГОВОРИТЬ НА ЗАЩИТЕ**
> Page cache — область памяти ядра со страницами файлов. Буферизованный ввод-вывод копирует данные между page cache и буфером процесса; изменённые страницы помечаются грязными и асинхронно сбрасываются на носитель (writeback). O_DIRECT предписывает передачу непосредственно между буфером процесса и устройством, минуя page cache, при выравнивании адреса, смещения и длины по логическому блоку устройства. O_DIRECT не гарантирует долговечности записи (для этого `fsync` или `O_SYNC`) и не отключает собственный кэш накопителя.

3.4 Сброс кэша перед запуском

```
sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

- `sync` — записать все грязные страницы на диск (выкинуть можно только чистые);
- `echo 3` — выкинуть чистые страницы page cache (1) и кэши каталогов и inode (2): 1 + 2 = 3.

Так каждый запуск начинается с **холодного кэша**, и состояние кэша одинаковое для всех конфигураций.

3.5 Вопросы

В: Что будет, если не выровнять буфер при O_DIRECT?
О: `read()`/`write()` вернёт `EINVAL`.

В: Зачем перечитывать блок перед записью?
О: Запись идёт целым блоком 512 Б, а менять надо 8 байт. Без перечитывания затёрлись бы соседние вершины.

В: Нужен ли read-версии drop_caches?
О: Для её времени — нет: с O_DIRECT она не пользуется page cache. Делается одинаково для обеих программ, чтобы методика была единой.

В: Почему sync перед drop_caches?
О: drop_caches выкидывает только чистые страницы; грязные сначала надо записать.

4 mmap

4.1 Что делает mmap

```
int fd = open("graph.bin", O_RDWR);
char *p = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
p[100] += 1;          /* это обращение к памяти, не системный вызов */
```

После `mmap()` **ничего не прочитано**. Ядро только записало: «диапазон адресов от `p` до `p + size` — это файл `graph.bin`». Таблица страниц для этого диапазона пустая.

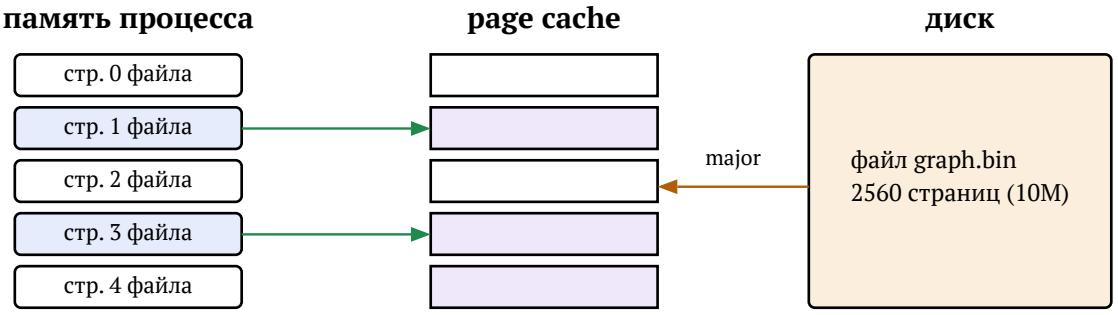


Рис. 8. Отображённый файл — это те же страницы page cache, вписанные в таблицу страниц процесса. Копирования нет.

Дальше:

1. программа обращается к `p[x]` → страницы нет в таблице → **page fault**;
2. ядро ищет страницу в page cache. Нет — читает с диска (**major**), обычно сразу несколько соседних страниц (**readahead**). Есть — просто отображает (**minor**);
3. ядро вписывает страницу в таблицу, инструкция повторяется;
4. все следующие обращения к этой странице — **со скоростью памяти**, без участия ядра.

4.2 Запись через mmap

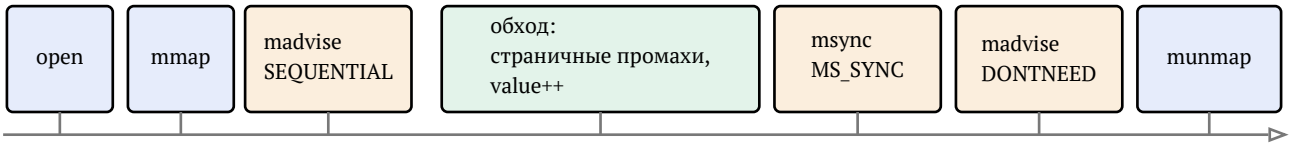
`MAP_SHARED` значит: изменения идут в общие страницы page cache и в итоге попадают в файл.

Чтобы знать, какие страницы изменились, ядро сначала отображает страницу **только для чтения**. Первая запись в неё вызывает ещё один (minor) page fault; ядро помечает страницу грязной и разрешает запись. Поэтому в режиме `--write` примерно **два промаха на страницу**: один на чтение, один на первую запись.

> **ПРИМЕР ИЗ МОИХ ЗАМЕРОВ**
> mmap-версия, 10M (2560 страниц): minor faults ≈ 2913 при «фоне» ≈ 220 от загрузки самой программы. 2913 − 220 ≈ 2700 ≈ одна-две записи на страницу плюс 208 major.

4.3 Что делает –no-cache у mmap

У mmap нет аналога `O_DIRECT`: отображённая память **и есть** страницы page cache. Обойти кэш нельзя. Поэтому в `graph_traverse_mmap.c` ключ `--no-cache` сделан тремя вызовами:



время → (оранжевое — то, что добавляет `--no-cache`)

Рис. 9. Последовательность вызовов mmap-версии с `--write --no-cache`.

Вызов	Что значит простыми словами
`madvise(MADV_SEQUENTIAL)`	**Совет ядру**: «читать буду подряд». Ядро в ответ при промахе подгружает побольше страниц вперёд. Ядро может совет и не учесть; на результат программы он не влияет — только на то, как ядро подгружает страницы.
`msync(MS_SYNC)`	«Запиши все изменённые страницы на диск **прямо сейчас**, и я подожду». **Синхронно** = программа стоит, пока запись не закончится. Нужно, чтобы время записи **вошло в замер**: иначе изменения остались бы в RAM, и mmap выглядела бы нечестно быстрой.
`madvise(MADV_DONTNEED)`	«Эти страницы мне больше не нужны». Ядро вычёркивает их из таблицы страниц процесса. **Сами страницы файла могут остаться в page cache** — ядро не обязано их выкидывать.

Аналогия с библиотекой: `MADV_SEQUENTIAL` — сказать библиотекарю «читаю подряд» (он может принести следующие тома заранее); `msync` — дождаться, пока он внесёт твои правки в каталог; `MADV_DONTNEED` — убрать закладки из своего блокнота, а книга остаётся на полке.

В справке самой программы так и написано: `--no-cache : advisory madvise only; mmap still uses the page cache`.

> **СТРОГО — МОЖНО ГОВОРИТЬ НА ЗАЩИТЕ**
> mmap(2) создаёт отображение области файла в адресное пространство процесса; страницы загружаются лениво, по страничным промахам. При MAP_SHARED изменения пишутся в общие страницы page cache. Отключить page cache для отображённого файла нельзя, поэтому `--no-cache` у mmap-версии реализован рекомендациями ядру (`MADV_SEQUENTIAL`, `MADV_DONTNEED`) и синхронным сбросом грязных страниц `msync(MS_SYNC)`, время которого входит в замер. Холодный кэш в моих замерах обеспечивается `drop_caches` перед каждым запуском.

4.4 Почему mmap быстрее в тысячи раз

На одной странице 4 КиБ помещается 4096 / 24 ≈ **170 вершин**. Цепочка рано или поздно проходит их все.

	read + O_DIRECT	mmap
Стоимость вершины	6 системных вызовов + 3 операции с диском ≈ 340 мкс	обращение к памяти — наносекунды
Стоимость страницы	170 × 3 = 510 операций с диском	1 промах (+1 на первую запись), диск — раз на ≈12 страниц
Запись на диск	на каждой вершине, блоком 512 Б	один раз в конце, `msync`, весь файл

10M целиком	≈ 1,3 млн операций с диском	≈ 208 чтений пачками + запись файла

4.5 А если бы не было drop_caches или –no-cache

	read-версия	**mmap-версия**
без `drop_caches`	ничего не меняется: O_DIRECT игнорирует page cache	страницы остались от прошлого запуска; все промахи minor, **чтений с диска ноль**
без `--no-cache`	работает через page cache: диск — раз на страницу, но на каждую вершину остаются системные вызовы и копирование ≈ 1–2 мкс	почти как сейчас, но без `msync`: запись уйдёт на диск в фоне, после завершения

> **ПРИМЕР ИЗ МОИХ ЗАМЕРОВ**
> Моя базовая часть лабы (26.09, тот же компьютер, файл 256 МиБ, запись, **с кэшем**, Rand): `graph_traverse` 15,8 с, из них sys 13,7 с; `graph_traverse_mmap` 1,1 с. Разница ≈ 15 раз — это цена системных вызовов. В варианте без кэша разница ≈ 2000 раз — к системным вызовам добавились походы на диск на каждой вершине.

4.6 Вопросы

В: Что такое «совет ядру»?
О: Вызов `madvise`: программа сообщает, как будет пользоваться памятью. Ядро может учесть или проигнорировать; результат программы от этого не меняется.

В: Что значит «синхронно записать»?
О: `msync(MS_SYNC)` возвращается только после того, как грязные страницы записаны на носитель; программа ждёт.

В: Можно ли через mmap обойти page cache?
О: Нет. Отображённая память — это страницы page cache.

В: Почему в режиме записи у mmap больше minor faults?
О: Страница сначала отображается только для чтения, первая запись вызывает промах, ядро помечает её грязной.

5 Программы и граф изнутри

5.1 Генерация графов

```
python3 ../util/graphgen.py -s 10M --seed 427 --topology chain \
        -b 0.5 --min-step-pages 2 -o data-variant/graph-rand-10M.bin
python3 ../util/graphgen.py -s 1M  --seed 427 --topology chain \
        -b 0.5 --min-step-pages 2 -o data-variant/graph-rand-1M.bin
```

`-s 10M`	размер файла 10 МиБ = 10 485 760 байт ⟶ 436 905 вершин
`--seed 427`	зерно генератора; одинаковое — графы различаются только размером
`--topology chain`	одна цепочка через все вершины, физический порядок случайный
`-b 0.5`	вероятность прыжка назад — половина прыжков к началу файла
`--min-step-pages 2`	прыжок не короче двух страниц (8 КиБ) — следующая вершина гарантированно на другой странице

Замеры идут на копиях `graph-rand-*.w.bin`, потому что `--write` меняет значения в файле.

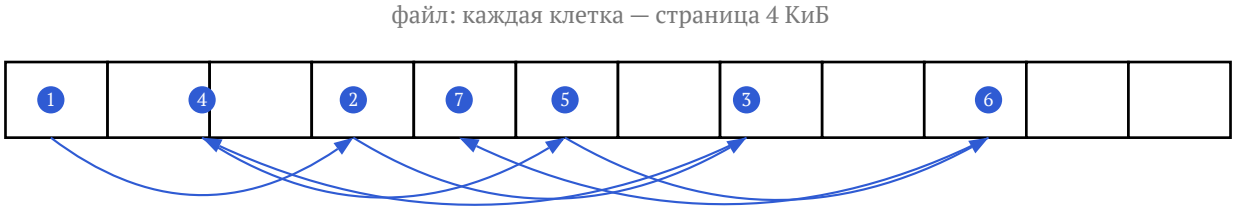


Рис. 10. Rand: цепочка 1 ⟶ 2 ⟶ 3 ⟶ … прыгает по файлу на расстояние не меньше двух страниц, вперёд и назад.

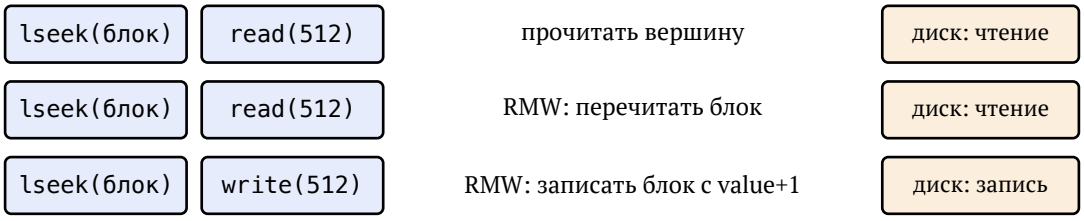
5.2 Цикл обхода read-версии

Из `src/graph_traverse.c` (сокращено):

```
while (1) {
    read_vertex(&file, current, record_size, &value, &degree, &child);
    if (write_mode)
        write_value(&file, current, record_size, value + 1);
    if (degree == 0) break;
    current = child;
}
```

`read_vertex` ⟶ `graph_read(offset, 24 байта)`, `write_value` ⟶ `graph_write(offset, 8 байт)`. С `O_DIRECT` обе идут через `graph_direct_transfer`, которая выравнивает по блоку.

5.3 Одна вершина в read-версии



итого на вершину: **6 системных вызовов, 3 операции с диском**, ≈ 3 засыпания процесса

Рис. 11. Что происходит на каждой вершине у read-версии с `--write --no-cache`.

Проверка по `strace -c` (1M, 43 689 вершин):

вызов	сколько раз	на вершину
`read`	88 745	2,03
`write`	43 691	1,00
`lseek`	132 433	3,03

Чуть больше двух чтений на вершину — это вершины на границе блоков (рисунок в главе 3).

Проверка по `/usr/bin/time -v` (среднее по серии, единицы по 512 Б):

счётчик	1M	10M	на вершину
File system inputs	90 656	889 328	≈ 2,04
File system outputs	43 697	436 921	1,00
Voluntary context switches	132 460	1 324 412	≈ 3,03

Всё сходится с картинкой: два чтения и одна запись на вершину, и на каждой операции процесс засыпает в ожидании диска (произвольное переключение контекста).

5.4 mmap-версия

На вершине — `node->value++` в памяти. Системные вызовы только по краям: `open`, `mmap`, `madvise`, `msync`, `madvise`, `munmap`.

счётчик (`time -v`)	1M (256 страниц)	10M (2560 страниц)
major faults	22	208
minor faults	490	2913
File system inputs × 512 Б	3928 ≈ 1,9 МиБ	22 360 ≈ 10,9 МиБ
File system outputs × 512 Б	2056 ≈ 1 МиБ	20 488 = 10 МиБ

- **major faults меньше, чем страниц:** readahead приносит ≈ 12 страниц за раз (2560 / 208);
- **inputs ≈ весь файл:** файл прочитан один раз (плюс сама программа и библиотеки);
- **outputs = весь файл:** каждая страница стала грязной, `msync` записал файл целиком.

5.5 Вопросы

В: Сколько операций с диском на вершину у read-версии?
О: Три: два чтения блока и одна запись.

В: Откуда 2,03, а не 2 чтения?
О: Около 4–5% вершин пересекают границу блока 512 Б, и их чтение занимает два блока.

В: Почему у mmap major faults меньше, чем страниц в файле?
О: Readahead: при промахе ядро читает сразу несколько соседних страниц.

В: Зачем копии `*.w.bin`?
О: Режим записи меняет `value`; исходный граф остаётся нетронутым.

6 Постановка эксперимента и инструменты

6.1 Паспорт системы

Машина	домашний ПК `tim-homie`, без виртуализации
ОС	Ubuntu 24.04, ядро 7.0, x86-64
CPU	Intel Core i5-11400F, 6 ядер / 12 потоков; физическое ядро N = логические CPU N и N+6
RAM	31 ГиБ; файлы 1 и 10 МиБ помещаются целиком
Диск	NVMe Kingston NV3 1 ТБ, логический блок 512 Б, ext4

6.2 Окружение: что сделал и что было бы без этого

Действие	**Зачем**	**Без этого**
`cpupower frequency-set -g performance`	частота максимальная базовая и не меняется	после перезагрузки стоял `powersave`: частота плавает от нагрузки
Turbo выключен (`no_turbo = 1`)	частота не зависит от нагрева	первые запуски быстрее последних
`taskset -c 2`	процесс не переезжает между ядрами; CPU 0 обслуживает пре-	миграции, лишний разброс

	рывания	
`nice -n -5`	приоритет выше фоновых задач	фоновые задачи могут вытеснять
`sync` + `drop_caches` перед каждым запуском	одинаковое холодное состояние кэша	mmap-версия читала бы из RAM
Firefox и окна обновлений приостановлены `kill -STOP`	не пишут на тот же диск, не грузят CPU	в серии 28.09 разброс был 68–79%
Интерливинг: каждый раунд — все 4 конфигурации в случайном порядке	медленный дрейф условий размазывается по всем	при блоках одна конфигурация целиком могла попасть в «плохой» период
Первые 2 раунда отброшены (решено заранее)	прогрев	—

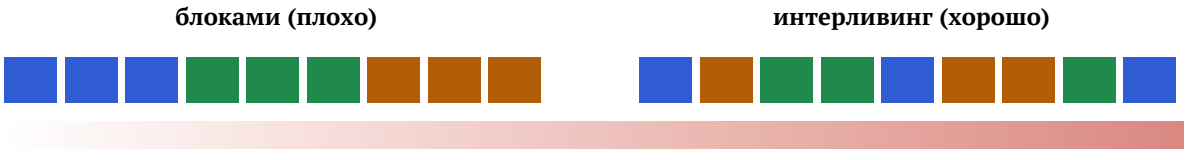


Рис. 12. При блоках «оранжевая» конфигурация целиком попадает в худший период. При интерливинге — все поровну.

6.3 Инструменты

Инструмент	Что показывает	Где использовал
таймер bash `$EPOCHREALTIME`	wall с точностью 1 мкс	серия — основная метрика
`/usr/bin/time -v`	user, sys, промахи, переключения контекста, объём ввода-вывода; wall только с точностью 10 мс	серия — сопутствующие счётчики
`strace -c`	какие системные вызовы и сколько	ознакомительно
`perf stat`	счётчики ядра и процессора: context-switches, page-faults, cache-misses	ознакомительно
`iostat -dx`	операции диска в секунду, задержка устройства `r_await/w_await`	параллельно серии — объяснение разброса

> **ВАЖНО**
> Почему два таймера. Время mmap-версии — десятки миллисекунд. У `/usr/bin/time` шаг 10 мс, поэтому в старой серии у mmap выходило `0,00` или `0,01` — это не измерение. Таймер bash меряет с точностью 1 мкс. Он включает запуск `time`, `taskset`, `nice` — это постоянная добавка ≈ 1–2 мс, одинаковая для всех конфигураций.

6.4 wall, user, sys

- **wall** — время по часам от старта до конца;
- **user** — время процессора в коде самой программы;
- **sys** — время процессора в ядре по просьбе программы (системные вызовы, промахи).

Если wall ≫ user + sys, процесс большую часть времени **ждал**.

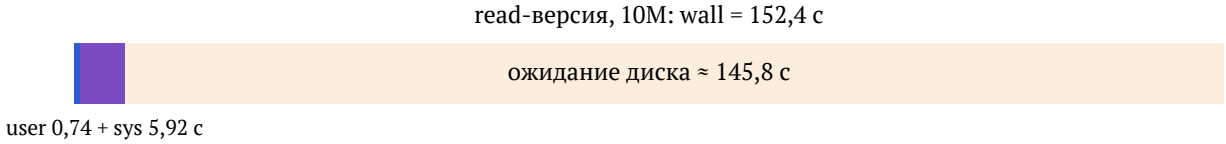


Рис. 13. 96% времени read-версии процесс спит в ожидании диска.

> **СТРОГО — МОЖНО ГОВОРИТЬ НА ЗАЩИТЕ**
> Произвольное (voluntary) переключение контекста происходит, когда процесс сам блокируется в ожидании ресурса, например завершения ввода-вывода; принудительное (involuntary) — когда планировщик вытесняет процесс. У read-версии с O_DIRECT около трёх произвольных переключений на вершину — по одному на каждую синхронную операцию с устройством.

6.5 Вопросы

В: Зачем taskset?
О: Чтобы процесс не мигрировал между ядрами: миграция сбрасывает кэши CPU и добавляет разброс.

В: Почему ядро 2, а не 0?
О: CPU 0 чаще обслуживает прерывания. Сосед по SMT у CPU 2 — CPU 8, он не нагружен.

В: Зачем интерливинг?
О: Если условия медленно меняются, при блоках одна конфигурация целиком попала бы в «плохой» период.

В: Почему strace не в финальной серии?
О: Он перехватывает каждый системный вызов и сильно замедляет read-версию — у неё их ≈ 6 на вершину.

В: Почему разрешено было остановить Firefox?
О: Это фоновая программа, не относящаяся к эксперименту; README требует завершить фоновые задачи. SIGSTOP приостанавливает процесс без потери состояния.

7 Статистика

7.1 Формулы

$\overline{x} = \frac{1}{N}\sum_{i=1}^{N} x_i$	среднее
$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \overline{x})^2}$	выборочное стандартное отклонение
$\mathrm{SE} = \frac{s}{\sqrt{N}}$	стандартная ошибка среднего
$\text{ДИ} = \overline{x} \pm t_{0.975;N-1} \cdot \mathrm{SE}$	95-процентный доверительный интервал
$N \approx \left(t \cdot \frac{s}{E}\right)^2$	сколько запусков нужно для полуширины ДИ, равной E

7.2 Разбор на моих числах: read-версия, 10M

Десять запусков после прогрева, секунды:

```
153,3  182,9  140,6  148,1  140,8  150,1  168,7  157,8  141,0  140,9
```

1. **Среднее:** сумма 1524,1 / 10 = **152,4 с**.
2. **Стандартное отклонение:** s = **14,1 с**. Знаменатель N − 1 = 9: среднее само посчитано по этим же точкам, одна степень свободы «потрачена».
3. **Стандартная ошибка:** SE = 14,1 / √10 = 14,1 / 3,162 = **4,45 с**.
4. **Квантиль Стьюдента:** t(0,975; 9) = **2,262**.
5. **ДИ:** 152,4 ± 2,262 · 4,45 = 152,4 ± **10,1 с** ⟶ [142,3; 162,5].

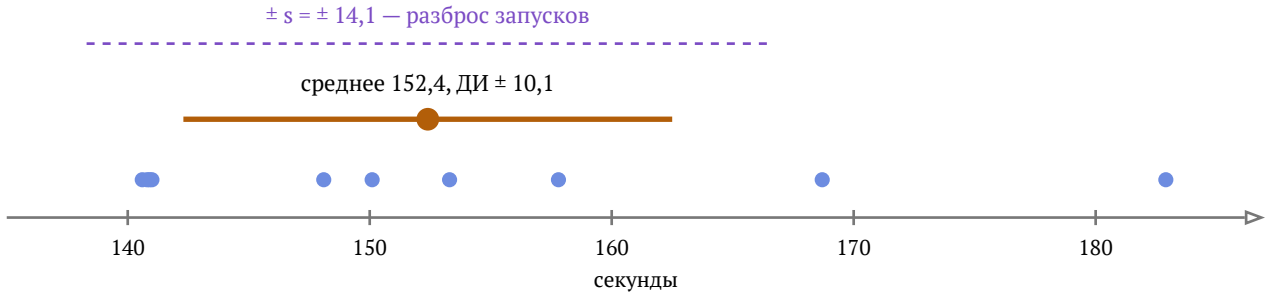


Рис. 14. Синие точки — запуски. Оранжевый — ДИ среднего. Фиолетовый — стандартное отклонение. ДИ уже, чем s, и сужается с ростом N; s — нет.

7.3 s и ДИ — разные вещи

- **s** — насколько разбросаны отдельные запуски. С ростом N **не уменьшается** — это свойство системы.
- **ДИ** — насколько точно мы знаем среднее. Сужается как $1/\sqrt{N}$: в 4 раза больше запусков — в 2 раза уже интервал.

7.4 Почему t, а не 1,96

Дисперсия заранее неизвестна, мы её оценили по тем же 10 точкам. Распределение Стьюдента это учитывает и даёт интервал шире. При N = 10 t = 2,262, при N = 30 — 2,045, при большом N стремится к 1,96.

7.5 Сравнение конфигураций

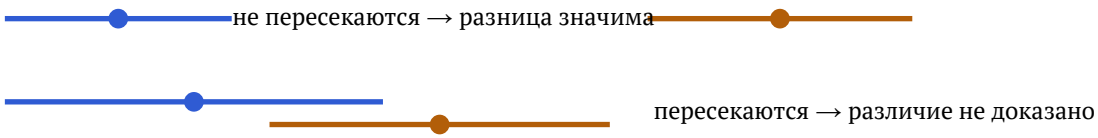


Рис. 15. Простой критерий из методички курса. Строже — t-тест Уэлча.

7.6 Сколько нужно запусков

Формула: $N \approx (t \cdot s / E)^2$, где E — желаемая полуширина ДИ.

Пример для read-10M, хочу E = 5% среднего = 7,6 с: $N \approx (2{,}262 \cdot 14{,}1 / 7{,}6)^2 \approx (4{,}19)^2 \approx$ **17**.

Для read-1M $s/\bar{x}$ гораздо больше (35%), и та же точность требует ≈ 256 запусков. Это не значит, что выводы неверны: различия между конфигурациями (в 10 и в сотни раз) много больше ширины интервалов.

7.7 Прогрев и выбросы

- **Прогрев:** решение «отбросить первые 2 раунда» принято до серии. Больше ничего не отбрасывал.
- **Выбросы** отбрасывают только по формальному правилу, принятому заранее (например, 1,5·IQR). Медленные запуски у меня — не ошибки, а реальное поведение диска (глава 8), поэтому они оставлены.
- Из-за «длинного хвоста» дополнительно указаны **медиана** и межквартильный размах: они устойчивее среднего.

7.8 Гистограмма и KDE



гистограмма: столбики по корзинам KDE: сумма «холмиков» на каждую точку

Рис. 16. Оба способа показывают форму распределения. У меня оно двугорбое — «быстрые» и «медленные» запуски.

- **Гистограмма** — точки раскладываются по корзинам, высота столбика = сколько попало. Форма зависит от ширины и положения корзин.
- **KDE** (оценка плотности ядром) — на каждую точку кладётся маленькая гауссиана, они суммируются. Гладкая кривая; форма зависит от ширины ядра (bandwidth).

Задание просит построить их, чтобы увидеть: распределение времени **не нормальное**, наивно считать его нормальным нельзя.

> **СТРОГО — МОЖНО ГОВОРИТЬ НА ЗАЩИТЕ**
> 95-процентный доверительный интервал — интервал, построенный процедурой, которая в 95% повторений эксперимента накрывает истинное математическое ожидание. При неизвестной дисперсии используется квантиль распределения Стьюдента с N − 1 степенями свободы. ДИ характеризует неопределённость оценки среднего и сужается как $1/\sqrt{N}$; стандартное отклонение характеризует разброс измерений и от N не зависит.

7.9 Вопросы

В: Что будет с ДИ, если увеличить N в 4 раза?
О: Сузится примерно в 2 раза.

В: A с s?
О: Останется примерно таким же.

В: Почему N − 1 в формуле s?
О: Среднее посчитано по той же выборке; деление на N − 1 даёт несмещённую оценку дисперсии.

В: Почему не выбросил медленные запуски?
О: Правила для выбросов заранее не было, а медленные запуски — реальное поведение диска, а не ошибка измерения.

В: Чем KDE лучше гистограммы?
О: Не зависит от границ корзин и даёт гладкую оценку плотности; зато зависит от ширины ядра.

8 Мои результаты

8.1 Серия

12 раундов × 4 конфигурации, порядок внутри раунда случайный, первые 2 раунда отброшены → **N = 10** на конфигурацию. Машинное время серии ≈ 35 минут.

Программа	Файл	Среднее, с	ДИ 95%, ±с	Медиана, с	Мин–макс, с	мкс / вершину
read	1M	14,82	3,75	11,66	11,06–25,34	339
read	10M	152,4	10,1	149,1	140,6–182,9	349
mmap	1M	0,035	0,034	0,014	0,012–0,151	0,81
mmap	10M	0,077	0,024	0,065	0,051–0,142	0,18

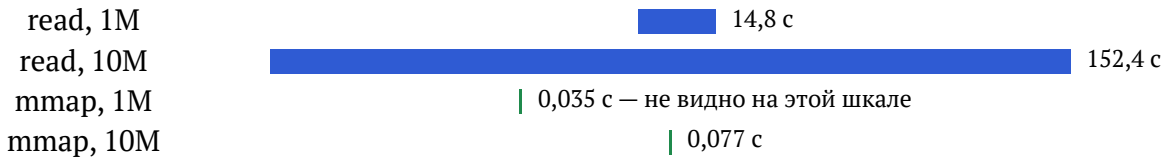


Рис. 17. Среднее время, линейная шкала. mmap-версия на этой шкале не видна.

8.2 Что видно

1. **read: 10M / 1M = 10,3 по средним.** Вершин в 10 раз больше. Время на вершину 339 и 349 мкс, их ДИ пересекаются → **время растёт пропорционально числу вершин**. Ожидаемо: без кэша каждая вершина стоит одинаково — три операции с диском.
2. **read: wall ≫ user + sys.** 10M: 152,4 с против 6,7 с — 96% времени ожидание диска.
3. **mmap быстрее read примерно в 420 раз на 1M и в 2000 раз на 10M.**
4. **mmap: 10M / 1M ≈ 2,2 по средним и 4,7 по медианам — не 10.** Время mmap-версии — это запуск процесса, ≈ 200 промахов с чтением пачками и `msync`, а не обход вершин. Постоянная часть (запуск ≈ 1–2 мс) и запись файла дают нелинейность.
5. **Разброс большой.** read-1M: запуски делятся на быстрые (≈ 11,1 с) и медленные (16–25 с), CV 35%.

8.3 Откуда разброс: диск

Параллельно серии работал `iostat`. В «медленные» периоды:

	быстрый период	медленный период
чтений в секунду (`r/s`)	≈ 8000	≈ 3200
задержка чтения устройства (`r_await`)	0,11 мс	0,30 мс
задержка записи (`w_await`)	0,01 мс	0,01 мс

Задержка растёт **на уровне устройства** — процессор, планировщик и фоновые программы ни при чём. Это поведение накопителя Kingston NV3. Запись быстрая (0,01 мс), потому что диск принимает её в свой внутренний буфер. Убрать это я не могу, поэтому медленные запуски оставлены в выборке, а разброс объяснён.

> **ВАЖНО**
> В старой серии 28.09 разброс был ещё больше (68–79%): к поведению диска добавлялся фоновый Firefox, а время mmap мерилось с шагом 10 мс. Поэтому серия переделана.

8.4 Достаточно ли N = 10

- для полуширины ДИ 5% от среднего нужно: read-10M — 17, read-1M — 256 запусков;
- для **выводов** хватает: ДИ 1M и 10M не пересекаются ни у одной программы, ДИ read и mmap отличаются на 2–3 порядка;
- увеличение N сузит интервалы как $1/\sqrt{N}$, но выводов не изменит.

8.5 Вопросы

В: Подтвердилась ли линейность у read-версии?
О: Да: отношение средних 10,3 при отношении числа вершин 10; время на вершину в пределах погрешности одинаковое.

В: Почему у mmap не в 10 раз?
О: Основное время — не обход, а постоянные издержки: запуск процесса, промахи, `msync`. Обход вершин в памяти почти бесплатный.

В: Откуда двугорбое распределение?
О: Задержка чтения у диска периодически растёт в ≈ 3 раза (`r_await` 0,11 → 0,30 мс по iostat).

9 Отчёт и защита

9.1 Что может попросить практик (README, раздел 8)

- **заново прогнать сбор**: генерация графа → сборка → `scripts/collect_variant.sh`;
- **объяснить каждое решение по окружению** — глава 6;
- **объяснить ДИ и что будет при другом N** — глава 7;
- **обосновать размер графа относительно RAM**: 10 МиБ ≪ 31 ГиБ; без `--no-cache` и `drop_caches` файл жил бы в page cache, и диск бы не участвовал. Размер не уменьшал: на чистой машине серия заняла ≈ 35 минут;
- **рассказать, как устроены обе программы и mmap** — главы 4–5.

9.2 Команды для защиты

```
ssh home
cd ~/projects/itmo-corp/study/os/os-course/lab/intro-exp

# сборка
mkdir -p out
gcc -O2 -o out/graph_traverse      src/graph_traverse.c
gcc -O2 -o out/graph_traverse_mmap src/graph_traverse_mmap.c

# окружение
sudo cpupower frequency-set -g performance

# одиночные запуски
time ./out/graph_traverse      --write --no-cache 1 data-variant/graph-rand-1M.w.bin
time ./out/graph_traverse_mmap --write --no-cache 1 data-variant/graph-rand-1M.w.bin

# короткая демонстрация сбора: 1 раунд, 1 итерация, ядро 2, только 1M
SIZES=1M ./scripts/collect_variant.sh /tmp/demo.csv 1 1 2
cat /tmp/demo.csv
```

9.3 Самые вероятные вопросы

В: Почему mmap настолько быстрее?
О: Страница 4 КиБ (≈ 170 вершин) загружается один раз, дальше доступ из памяти. read-версия с O_DIRECT на каждую вершину делает три операции с диском.

В: Что значит –no-cache для каждой программы?
О: read — `O_DIRECT`, мимо page cache. mmap — только `madvise`-рекомендации и `msync`; page cache используется.

В: Честное ли сравнение read и mmap?
О: Нет в смысле одинаковой семантики: один и тот же ключ означает разное. Это и есть один из выводов.

В: Почему wall ≫ user + sys?
О: Процесс спит в ожидании диска; это видно по ≈ 3 произвольным переключениям контекста на вершину.

В: Почему 10M и 1M, а не меньше?
О: README разрешает уменьшать размер, если серия слишком долгая. На чистой машине обход 10M ≈ 150 с, серия ≈ 35 минут — уменьшать не понадобилось.

В: Что изменилось бы на HDD?
О: Каждая операция ≈ 5–10 мс вместо 0,1 мс: read-версия на 10M шла бы около часа. Для mmap разница меньше: чтения пачками.

В: Насколько интрузивны инструменты?
О: strace сильно замедляет read-версию (перехват ≈ 6 вызовов на вершину), поэтому только ознакомительно. В серии — таймер bash и `/usr/bin/time`.

В: Откуда минимум в 2 страницы в генераторе?
О: `--min-step-pages 2`: без него часть переходов оставалась бы на той же странице, и доступ не был бы случайным.